

Важни указания

В Startup добавяме този ред код за използване на MVC – model-view-controller, за да изключим с false използването на EndpointRouting, който изписва “Hello world!”.

```
public void ConfigureServices(IServiceCollection services)
{
    services.AddMvcCore(option => option.EnableEndpointRouting = false);
}
```

Понеже видяхме, че всичко е в слоеве middleware и ако не намери един изход, прехвърля на следващия. Ако не намери index файл, който да се стартира by default -> прескача в Endpoint и изписва “Hello world” или хвърля грешка – каквото му кажем. Това е принципът на pipeline – времевата линия.

```
app.UseEndpoints(endpoints =>
{
    endpoints.MapGet("/", async context =>
    {
        throw new Exception("Error message!");
        await context.Response.WriteAsync("Hello world!");
        // await context.Response.WriteAsync(System.Diagnostics.Process.GetCurrentProcess().Name);
        // await context.Response.WriteAsync(_config ["MyKey"]);
        // await context.Response.WriteAsync(" + Hello from 2-nd Middleware!");
        //await context.Response.WriteAsync("MW3");
        //logger.LogInformation("MW3: Request handled and response produced");
    });
});
```

Т.е., понеже ние правим сайт, вече не искаме текст да се отпечата в браузъра, затова Endpoints вече няма да ни бъде необходим, изключваме го с false и ще използваме вместо него MVC – с модел, изглед, контролер. По-нагоре съм маркирала думата Core не случайно – може да се използва app.UseMvc или appUseMvcCore – второто е по-добре, защото Core съдържа повече възможности. Когато го направим, би трябвало автоматично да се включи библиотеката:

```
using Microsoft.AspNetCore.Http;
using Microsoft.AspNetCore.Mvc;
using Microsoft.Extensions.Configuration;
```

Тя е с по-светъл цвят, защото е добавена по-късно и нека обърнем внимание, че библиотеките се подреждат по азбучен ред. Ако не се добави автоматично и се наложи ние да я напишем, ако не се съобразим къде правилно да я поставим по азбучен ред, тя няма да бъде разпозната и ще се хвърлят грешки в MVC кода. Принципно, тези грешки се редактират лесно, като при червеното

подчертаване се натисне Ctrl + . и се потвърди корекцията с Enter – така би трябвало автоматично да си добави необходимата библиотека на правилното място.

Да дообясним и следващите редове код.

if- ако, ако сме в среда за разработка, нека се използва DeveloperExceptionPage – страницата за грешки, като всяка грешка Error се разглежда като прихванато изключение Exception. Така, ако всички грешки се прихванат като изключения, грешки изобщо няма да има. Обикновено те са някакви крайни гранични случаи.

`app.UseDeveloperExceptionPage();`

това е напълно достатъчен ред код. Използваме го с `app.Use` пред вградения метод `DeveloperExceptionPage()` – скобите показват именно, че е метод.

Ако искаме да променим неговата настройка, той има вградени възможности като например `SourceCodeLineCount` – които се извикват с точка. За да стигнем до тях, обаче, трябва да извикаме настройките `Options`. Това става, като залепим думата `Options` в края на метода.

Методите винаги започват с главна буква, а променливите с малка!!!

`DeveloperExceptionPageOptions` – така влизаме в настройката.

Задаваме име на новата настройка по избор, то може да бъде всякакво, но стандартно то е същото като настройката, просто с малка буква `developerExceptionPageOptions` и казваме, че това ще бъде нова настройка от този тип на този метод. Все едно имаме присвояване от по-сложен тип, аналогично на

`ВграденМетодOptions име = new ВграденМетодOptions();`

`име.достъпниНастройки ;`

`app.UseВграденМетод(име);`

Т.е. на самостоятелния метод `app.Use___()`; който работи самостоятелно, вкарваме името на променливата, която той да използва.

Това е целият принцип във всички подобни настройки.

```
public void Configure(IApplicationBuilder app, IWebHostEnvironment env, ILogger<Startup> logger)
{
    bool т.е. true/false
    if (env.IsDevelopment())
    {
        // app.UseDeveloperExceptionPage();
        Настройка
        DeveloperExceptionPageOptions developerExceptionPageOptions = new DeveloperExceptionPageOptions();
        developerExceptionPageOptions.SourceCodeLineCount = 10;
        app.UseDeveloperExceptionPage(developerExceptionPageOptions);
    }
}
```

излишно, но не пречи - за MW1,2,3,3,2,1

име = нова Настройка () ;

вградена възможност

име

Името на променливата може да бъде всякакво.

```
if (env.IsDevelopment())  
{  
    DeveloperExceptionPageOptions dEPO = new DeveloperExceptionPageOptions();  
    dEPO.SourceCodeLineCount = 10;  
    app.UseDeveloperExceptionPage(dEPO);  
    // app.UseDeveloperExceptionPage();  
}
```

вместо дългото име developerExceptionPageOptions
демонстрирам кратко по избор dEPO

точката дава достъп до вградени възможности

извикваме името (вкарваме в скобите)

Можем дори да създадем няколко променливи (по 2 реда), но ще се изпълни само онази от тях, която се извика в дадената част от кода с app.Use__(име);

```
if (env.IsDevelopment())  
{  
    // app.UseDeveloperExceptionPage();  
    DeveloperExceptionPageOptions dEPO = new DeveloperExceptionPageOptions();  
    dEPO.SourceCodeLineCount = 10;  
  
    DeveloperExceptionPageOptions dEPO1 = new DeveloperExceptionPageOptions();  
    dEPO1.SourceCodeLineCount = 11;  
  
    DeveloperExceptionPageOptions dEPO2 = new DeveloperExceptionPageOptions();  
    dEPO2.SourceCodeLineCount = 12;  
  
    app.UseDeveloperExceptionPage(dEPO1);  
}
```

можем да създадем няколко променливи с настройки

ще се изпълни онази, която се извика в скобите при Use

```
// app.UseDeveloperExceptionPage();
```

```
DeveloperExceptionPageOptions a = new DeveloperExceptionPageOptions();  
a.SourceCodeLineCount = 5;
```

```
DeveloperExceptionPageOptions b = new DeveloperExceptionPageOptions();  
b.SourceCodeLineCount = 10;
```

```
DeveloperExceptionPageOptions c = new DeveloperExceptionPageOptions();  
c.SourceCodeLineCount = 20;
```

```
app.UseDeveloperExceptionPage(a);
```

```
// many rows code
```

```
app.UseDeveloperExceptionPage(b);
```

Това е универсален принцип, като можем на самите настройки да влизаме в поднастройки с още извиквания с точка:

```
DefaultFilesOptions defaultFilesOptions = new DefaultFilesOptions();
defaultFilesOptions.DefaultFileNames.Clear();
defaultFilesOptions.DefaultFileNames.Add("about.html");

app.UseDefaultFiles(defaultFilesOptions);

// app.UseDefaultFiles();
app.UseStaticFiles();
```

Можем за ориентир да сменим името вместо defaultFilesOptions да напишем променливата като dFO:

```
DefaultFilesOptions dFO = new DefaultFilesOptions();
dFO.DefaultFileNames.Clear();
dFO.DefaultFileNames.Add("about.html");

app.UseDefaultFiles(dFO);

// app.UseDefaultFiles();
```

Накрая си закоментираме излишните неща, колкото да помним, че сме ги учили като възможности и си оставяме само базовия ред, който ни трябва за извикването с Use:

```
public void Configure(IApplicationBuilder app, IWebHostEnvironment env, ILogger<Startup> logger)
{
    if (env.IsDevelopment())
    {
        app.UseDeveloperExceptionPage();

        /* DeveloperExceptionPageOptions developerExceptionPageOptions = new DeveloperExceptionPageOptions();
        developerExceptionPageOptions.SourceCodeLineCount = 10;
        app.UseDeveloperExceptionPage(developerExceptionPageOptions); */
    }

    /* DefaultFilesOptions defaultFilesOptions = new DefaultFilesOptions();
    defaultFilesOptions.DefaultFileNames.Clear();
    defaultFilesOptions.DefaultFileNames.Add("about.html");

    app.UseDefaultFiles(defaultFilesOptions); */

    app.UseDefaultFiles();
    app.UseStaticFiles();
}
```

Казахме, че е важна последователността: първо DefaultFiles за index.html, после StaticFiles за картинки.

```
app.UseDefaultFiles();
app.UseStaticFiles();
```

```
// app.UseDirectoryBrowser();
```

```
/*| FileServerOptions fileServerOptions = new FileServerOptions();
   fileServerOptions.DefaultFilesOptions.DefaultFileNames.Clear();
   fileServerOptions.DefaultFilesOptions.DefaultFileNames.Add("page1.html");
   app.UseFileServer(fileServerOptions); */
```

DirectoryBrowser е с много голяма мощ и пренебрегва дори грешки, като показва дървовидната структура на папките – закоментираме го, т.к. не искаме да си разглеждаме в браузъра папки и файлове, а да зареждаме интернет страници за сайт. Принципно го използваме само, когато нещо не се получава и искаме да проверим име и местоположение на файл.

Следващия коментар също демонстрира принципът, който обясних по-нагоре – може вместо fileServerOptions да напишем навсякъде просто fso и пак ще работи.

```
app.UseMvc(rb =>
{
    rb.MapRoute(
        name: "default",
        template: "{controller}/{action}/{id?}",
        defaults: new {controller = "home", action = "index"});
});

app.UseRouting();
```

Понеже започнахме от Empty версия .Net 5.0 и си пишем всичко сами, задаваме и следващите настройки, където id е с въпросителна, т.к. може да отсъства от адресирането – по подразбиране ще търси /home/index, а шаблонът е /home/privacy или някакво друго действие, като id не е задължително – може да го има като в home/edit/5 – да редактираме id = 5, или за delete, update, create... но ако в самата страница няма id, то не е необходимо да присъства в адреса на URL.

По този начин зададено, ще търси страница index.html, но за да се зареди тя, вече трябва контролер, който поне да каже return някаква стойност или view();